HEALTH INSURANCE MANAGEMENT SYSTEM

SOFTWARE REQUIREMENTS SPECIFICATION

ISO/IEC/IEEE 29148:2018 Compliant | Version 1.0 – Final

- Koushik Gaddam

Document Version	1.0 – Final Release
Status	Approved
Prepared By	System Architect & Technical Team
Organization	Health Insurance Inc.
Date of Issue	March 12, 2026
Confidentiality	Internal – Confidential

© 2025 Health Insurance Inc. All Rights Reserved.

Table of Contents

1. Introduction, Scope & Overview 3
2. Stakeholder Analysis & Use Cases 4
3. Functional Requirements – Customer Module 5
4. Functional Requirements – Staff & Administrator Modules 6
5. Premium Calculation Engine 7
6. Policy Lifecycle – State Machine Specification 8
7. Claim Lifecycle – State Machine Specification 10
8. Database Schema & Relational Design 12
9. Security Architecture & API Contracts 14
10. Frontend Architecture – Angular Design 17
11. Audit Log Dictionary & Invoice Generation 19
12. Testing, QA & Non-Functional Requirements 21
13. Deployment, Scalability & Future Roadmap 23
14. Glossary, Acronyms & Revision History 25

Chapter 1: Introduction, Scope & Overview

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional, technical, and non-functional requirements for the Health Insurance Management System (HIMS). It serves as the authoritative baseline between stakeholders and the development team, structured in accordance with ISO/IEC/IEEE 29148:2018.

1.2 Product Description

HIMS is a full-stack web application built on Clean Architecture using .NET 8 and Angular 17. It manages the complete insurance lifecycle — from quote calculation and policy application through to claim settlement and staff management — across four role-segregated portals.

Architecture Layer	Responsibility	Technology
Domain Layer	Core entities, enums, repository interfaces; zero external dependencies	C# Domain Models
Application Layer	Business logic, services, DTOs, use-case orchestration	.NET Services, AutoMapper
Infrastructure Layer	EF Core DbContext, repositories, PDF generation	SQL Server, QuestPDF
API Layer	Controllers, JWT middleware, CORS, global error handling	.NET 8 Web API, JWT Bearer

1.3 Technology Stack

Component	Technology	Version
Backend Runtime	.NET Web API (Clean Architecture)	.NET 8 LTS
ORM / Data Access	Entity Framework Core	EF Core 8
Database	Microsoft SQL Server	2019+
Frontend	Angular (Standalone Components)	Angular 17+
Authentication	JWT Bearer / BCrypt	RFC 7519 / BCrypt v2
PDF Generation	QuestPDF	v2.x
Unit Testing	XUnit + Moq	Latest Stable

1.4 System Constraints

- All monetary fields use DECIMAL(18,2) to prevent floating-point rounding errors.
- JWT tokens carry a mandatory 2-hour sliding expiration.
- Document uploads restricted to PDF, JPG, PNG; maximum 5 MB per file.
- Claim deductions execute within atomic EF Core transactions (ACID compliance).
- Role segregation must prevent cross-role portal access under all session configurations.

1.5 References

- ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering Requirements

- .NET 8 Web API Documentation – Microsoft | Angular 17 – Google
- OWASP Web Application Security Guidelines v4.2

Chapter 2: Stakeholder Analysis & Use Cases

2.1 Stakeholder Registry

Stakeholder	Role	Primary Interests
Customer (Insured)	End user; policyholder	Policy status, coverage balance, claim outcomes, invoice access
Insurance Agent	Staff; application reviewer	Pending queue, document verification, commission tracking
Claims Officer	Staff; claim adjudicator	Settlement queue, coverage sufficiency, audit compliance
System Administrator	Super-user	Staff management, plan configuration, global audit access, revenue KPIs
Compliance Officer	External auditor	Audit trail completeness, RBAC enforcement, data protection

2.2 Use Case Summary

UC ID	Use Case	Actor	Precondition
UC-01	Calculate Insurance Premium Quote	Customer (Anonymous)	Plan templates exist
UC-02	Submit Policy Application + Documents	Customer (Registered)	Quote generated; user authenticated
UC-03	Process Payment & Activate Policy	Customer (Registered)	Quote approved by Agent
UC-04	Submit Medical Claim	Customer (Registered)	Active policy; sufficient coverage
UC-05	Verify Policy Application Documents	Insurance Agent	Policy in PendingReview; Agent assigned
UC-06	Approve / Reject Policy Application	Insurance Agent	ALL documents in Verified status
UC-07	Adjudicate Medical Claim	Claims Officer	Claim PendingApproval; Officer assigned
UC-08	Onboard New Staff Member	Administrator	Admin authenticated session
UC-09	Create / Edit Plan Template	Administrator	Admin authenticated session
UC-10	Generate & Store PDF Invoice	System (Automated)	Payment transaction confirmed

2.3 Policy Application Flow (UC-02) – End-to-End

The policy application lifecycle is the most complex multi-actor flow in the system:

1. Customer selects a plan (Silver/Gold/Platinum) and enters age; PremiumService calculates quote.
2. Customer submits application — PremiumQuote record created with IsConvertedToPolicy = 0 (PendingReview).
3. Customer uploads mandatory identity documents (Aadhar/PAN) to DocumentVault.

4. Admin assigns the application to an Insurance Agent based on plan type.
5. Agent reviews each document individually, setting status to Verified or Rejected.
6. Document Gate: if all Verified → policy Approved (status=1); if any Rejected → auto-Rejected (status=2).
7. On approval, customer completes payment; PaymentController generates PDF invoice, activates policy (IsPaid=true).

Chapter 3: Functional Requirements – Customer Module

3.1 Authentication & Registration

ID	Requirement	Priority
FR-AUTH-01	Register new users with BCrypt-hashed passwords; Customer role forced at point of creation.	Must Have
FR-AUTH-02	Return signed JWT on login containing sub (UserID), email, and role claims.	Must Have
FR-AUTH-03	JWT tokens expire after 2 hours; expired tokens return HTTP 401.	Must Have
FR-AUTH-04	Reject login for accounts marked IsActive = false without exposing reason.	Should Have

3.2 Premium Quotation

ID	Requirement	Priority
FR-QUOT-01	Calculate monthly premium: Base × (1 + (Age÷10 × 0.10)) × TierMultiplier.	Must Have
FR-QUOT-02	Apply tier multipliers: Silver 1.0×, Gold 1.2×, Platinum 1.5×.	Must Have
FR-QUOT-03	CoverageAmount = MonthlyPremium × 120 × TierMultiplier.	Must Have
FR-QUOT-04	Persist quote as PremiumQuote with IsConvertedToPolicy = 0.	Must Have
FR-QUOT-05	Quotation calculation accessible without authentication (anonymous access).	Must Have

3.3 Document Upload

ID	Requirement	Priority
FR-DOC-01	Accept document types: Aadhar, PAN, Bill, MedicalReport, Invoice.	Must Have
FR-DOC-02	Enforce file type restriction: .pdf, .jpg, .png only.	Must Have
FR-DOC-03	Enforce maximum file size: 5 MB per document.	Must Have
FR-DOC-04	Documents stored in DocumentVault with RelatedEntityType linking to Policy or Claim.	Must Have
FR-DOC-05	Document status defaults to Pending on upload.	Must Have

3.4 Claims Submission

ID	Requirement	Priority
FR-CLM-01	Allow customers to submit claims against active policies or active converted quotes.	Must Have
FR-CLM-02	Claim submission requires: ClaimAmount, Reason text, and at least one supporting document.	Must Have

ID	Requirement	Priority
FR-CLM-03	Display claim history with color-coded status badges (PendingApproval/Approved/Rejected).	Must Have
FR-CLM-04	Customers cannot view or modify other customers' claims under any session.	Must Have

Chapter 4: Functional Requirements – Staff & Administrator Modules

4.1 Insurance Agent Module

ID	Requirement	Priority
FR-AGT-01	Agent inbox shows only applications assigned to the agent's designated plan templates.	Must Have
FR-AGT-02	Agent can set each document status individually: Pending → Verified or Rejected.	Must Have
FR-AGT-03	Document Gate: policy approval proceeds only if ALL documents are Verified.	Must Have
FR-AGT-04	Auto-reject policy application if ANY document is set to Rejected.	Must Have
FR-AGT-05	Auto-calculate and log agent commission at 5% of MonthlyPremium on policy activation.	Must Have
FR-AGT-06	Agent views commission history with Pending and Paid status tracking.	Should Have

4.2 Claims Officer Module

ID	Requirement	Priority
FR-OFF-01	Officer portal shows all PendingApproval claims assigned to the officer.	Must Have
FR-OFF-02	Officer has access to DocumentVault entries linked to each claim for evidence review.	Must Have
FR-OFF-03	Deduction Gate: claim approval blocked if ClaimAmount > CoverageAmount.	Must Have
FR-OFF-04	On approval, atomically deduct ClaimAmount from policy's CoverageAmount.	Must Have
FR-OFF-05	Deduction and audit log write execute within a single EF Core transaction.	Must Have

4.3 Administrator Module

ID	Requirement	Priority
FR-ADM-01	Admin is the only role authorized to invoke AddStaff endpoint.	Must Have
FR-ADM-02	Admin creates Plan Templates with PlanName, BasePremium, and TierMultipliers.	Must Have
FR-ADM-03	Admin assigns/reassigns Agents to plan templates and Officers to claim queues.	Must Have
FR-ADM-04	Admin has unfiltered read access to the complete PolicyActionLog.	Must Have
FR-ADM-05	Admin dashboard displays: Total Active Policies, MRR, Claim Payout Liquidity, Unpaid Commission Liability.	Must Have

ID	Requirement	Priority
FR-ADM-06	Admin soft-deletes staff/customer accounts via IsActive flag (no hard deletes).	Should Have

Chapter 5: Premium Calculation Engine

5.1 Engine Architecture

The Premium Calculation Engine (PremiumService.cs) is a Transient-scoped service in the Application Layer, ensuring a fresh instance per request with no state contamination between concurrent calculations. It exposes a single public calculation method consuming the admin-configured plan template for base values.

5.2 Calculation Formula

```
Monthly Premium = Base Premium × (1 + (Age ÷ 10) × 0.10) × Tier Multiplier
Coverage Amount = Monthly Premium × 120 × Tier Multiplier
```

5.3 Tier & Boundary Value Reference

Plan	Tier Mult.	Age 18 | Base $500	Age 45 | Base $500	Age 65 | Base $500	Coverage Formula
Silver	1.0×	$550 / mo	$700 / mo	$800 / mo	Prem × 120
Gold	1.2×	$660 / mo	$840 / mo	$960 / mo	Prem × 120 × 1.2
Platinum	1.5×	$825 / mo	$1,050 / mo	$1,200 / mo	Prem × 120 × 1.5

5.4 Agent Commission Calculation

Upon policy activation (triggered by confirmed payment), the system auto-computes and persists the agent's commission in AgentCommissionLog at a fixed rate of 5% of the activated policy's MonthlyPremium. Commission entries default to Status = 'Pending' until manually disbursed and marked 'Paid' by an Administrator.

```
Agent Commission = Monthly Premium × 0.05
```

5.5 Dashboard Analytics (DashboardService.cs)

The admin dashboard aggregates cross-entity metrics using No-Tracking EF Core queries:

KPI	Source Tables	Calculation Logic
Total Active Policies	Policies + PremiumQuotes	COUNT where Status=Active AND IsConvertedToPolicy=1
Monthly Recurring Revenue	Policies + PremiumQuotes	SUM(MonthlyPremium) from all active sources
Claim Payout Liquidity	Claims	SUM(ClaimAmount) where Status = 'Approved'
Unpaid Commission Liability	AgentCommissionLog	SUM(EarnedAmount) where Status = 'Pending'

Chapter 6: Policy Lifecycle – State Machine Specification

6.1 State Transition Matrix

The Policy Application follows a five-state machine with enforced transitions. Invalid transitions are rejected at the service layer; every valid transition is recorded in PolicyActionLog.

Current State	Triggering Event	Next State	Actor	Gate Condition
[None]	Customer submits quote application	PendingReview (0)	Customer	Authenticated; quote calculated
PendingReview	Admin assigns to Agent	PendingReview (0)*	Admin	Agent exists with matching plan
PendingReview	Agent approves all documents	Approved (1)	Agent	ALL docs Verified; zero Rejected
PendingReview	Any document rejected	Rejected (2)	Agent / System	≥1 document in Rejected status
Approved (1)	Customer completes payment	Active / Paid	Customer + System	Payment gateway confirmed
Active	Expiry date reached	Inactive	System (Scheduled)	ExpiryDate < current UTC
Rejected (2)	—	[Terminal]	N/A	No further transitions

6.2 State Descriptions

Draft / Quotation

Prospect calculates premium (anonymous or authenticated). No persistent DB record yet. No obligations on either party.

PendingReview (IsConvertedToPolicy = 0)

Application formally submitted with documents uploaded. Awaiting Admin assignment and Agent document review. All documents default to Pending status.

Approved (IsConvertedToPolicy = 1)

All documents verified by Agent. Awaiting customer payment. Policy not yet active; coverage not yet available.

Active (IsPaid = true)

Payment confirmed. PDF invoice generated and stored in DocumentVault. Policy live on customer dashboard with real-time CoverageAmount display.

Rejected (IsConvertedToPolicy = 2)

Terminal state. Agent found forged or invalid documents. No further transitions permitted. Refund/closure handled through admin channels.

6.3 Document Gate Logic (StaffController.VerifyPolicy)

This is the most critical business logic guard in the policy workflow:

8. Retrieve all DocumentVault entries linked to the target PremiumQuoteId.

9. If any entry has Status = 'Rejected' → immediately set application to Rejected (2) and write audit log.
10. If any entry has Status = 'Pending' → return HTTP 400: 'All documents must be reviewed before approval.'
11. If all entries are Verified → approve application; set IsConvertedToPolicy = 1.
12. Write PolicyActionLog: ActionType = 'PolicyVerification', OldValue, NewValue, PerformedByUserId, Timestamp.

Chapter 7: Claim Lifecycle – State Machine Specification

7.1 State Transition Matrix

Current State	Triggering Event	Next State	Actor	Gate Condition
[None]	Customer submits medical claim	PendingApproval	Customer	Active policy; amount > 0; docs uploaded
PendingApproval	Officer approves; coverage sufficient	Approved	Officer + System	ClaimAmount ≤ CoverageAmount
PendingApproval	Officer rejects or coverage insufficient	Rejected	Officer / System	ClaimAmount > CoverageAmount OR fraud flags
Approved	—	[Terminal]	N/A	Coverage deducted; audit logged
Rejected	—	[Terminal]	N/A	No deduction; customer may appeal via admin

7.2 Deduction Gate Logic (StaffController.DecideClaim)

The DecideClaim endpoint is a safety-critical operation modifying the policy's remaining CoverageAmount. The following sequence executes upon officer approval:

13. Retrieve the Claim record with its linked Policy or PremiumQuote parent.
14. Coverage check: if Claim.ClaimAmount > Policy.CoverageAmount → HTTP 400: 'Insufficient coverage balance.' Abort.
15. Begin implicit EF Core transaction via SaveChangesAsync.
16. Set Claim.Status = 'Approved'.
17. Compute: Policy.CoverageAmount = Policy.CoverageAmount − Claim.ClaimAmount.
18. Write PolicyActionLog: ActionType = 'ClaimApprovalDeduction'; OldValue = previous coverage; NewValue = updated coverage.
19. Call SaveChangesAsync() — commits atomically. If audit log write fails, entire transaction rolls back.
20. Return HTTP 200 with updated claim and coverage data.

7.3 Customer Claim Dashboard

The GetMyClaims endpoint returns all claims associated with a customer's policies in reverse chronological order, showing ClaimAmount, Status (color-coded badge), submission date, and Reason text. The view is read-only post-submission; modifications require administrative escalation. The customer's live dashboard also displays the remaining CoverageAmount in real time after each approved claim deduction.

Chapter 8: Database Schema & Relational Design

8.1 Relational Design Principles

The schema is managed by EF Core 8 code-first migrations against SQL Server 2019+. All parent-child relationships use DeleteBehavior.Restrict to prevent accidental cascade deletions of customer or claim history. All monetary columns are explicitly configured to DECIMAL(18,2) via Fluent API.

8.2 Entity Data Dictionary

Table: Users

Column	Type / Constraints	Description
Id	INT, PK, IDENTITY	Auto-incremented primary key
FullName	NVARCHAR(200), NOT NULL	Legal full name of the user
Email	NVARCHAR(256), NOT NULL, UNIQUE	Login email address
PasswordHash	NVARCHAR(MAX), NOT NULL	BCrypt v2 hashed password
Role	NVARCHAR(50), NOT NULL	Enum: Customer, Agent, ClaimsOfficer, Admin
IsActive	BIT, NOT NULL, DEFAULT 1	Soft delete flag

Table: PremiumQuotes

Column	Type / Constraints	Description
Id	INT, PK, IDENTITY	Auto-incremented primary key
UserId	INT, FK → Users.Id RESTRICT	Owning customer
QuoteReference	NVARCHAR(20), UNIQUE	System-generated ref (QUO-XXXX)
SelectedPlanName	NVARCHAR(100), NOT NULL	Silver / Gold / Platinum
CalculatedMonthlyPremium	DECIMAL(18,2), NOT NULL	Risk-adjusted monthly premium
CoverageAmount	DECIMAL(18,2), NOT NULL	Remaining coverage balance
IsConvertedToPolicy	INT, NOT NULL, DEFAULT 0	0=Pending, 1=Approved, 2=Rejected
IsPaid	BIT, NOT NULL, DEFAULT 0	Payment confirmation flag
AgentId	INT, FK → Users.Id, NULLABLE	Assigned agent reference
ExpiryDate	DATETIME2, NULLABLE	Policy expiration (UTC)

Table: Claims

Column	Type / Constraints	Description
Id	INT, PK, IDENTITY	Auto-incremented primary key
UserId	INT, FK → Users.Id RESTRICT	Customer who filed the claim
PolicyId	INT, FK → Policies.Id, NULLABLE RESTRICT	Legacy policy reference
PremiumQuoteId	INT, FK → PremiumQuotes.Id, NULLABLE RESTRICT	Quote-based policy reference
ClaimAmount	DECIMAL(18,2), NOT NULL	Requested monetary amount
Reason	NVARCHAR(MAX), NOT NULL	Medical justification text
Status	NVARCHAR(50), NOT NULL	PendingApproval / Approved / Rejected
SubmittedAt	DATETIME2, NOT NULL	UTC submission timestamp

Table: DocumentVault

Column	Type / Constraints	Description
Id	INT, PK, IDENTITY	Auto-incremented primary key
FileName	NVARCHAR(300), NOT NULL	Original file name
FileData	VARBINARY(MAX), NOT NULL	Binary file content
ContentType	NVARCHAR(100), NOT NULL	MIME type
DocumentType	NVARCHAR(50), NOT NULL	Aadhar / PAN / Bill / MedicalReport / Invoice
Status	NVARCHAR(50), DEFAULT 'Pending'	Pending / Verified / Rejected
RelatedEntityType	NVARCHAR(50), NOT NULL	Discriminator: Policy or Claim
RelatedEntityId	INT, NOT NULL	FK to parent entity

Table: PolicyActionLog

Column	Type / Constraints	Description
Id	INT, PK, IDENTITY	Log entry identifier (append-only)
ActionType	NVARCHAR(100), NOT NULL	Event category (see Audit Dictionary, Ch. 11)
RelatedEntityId	INT, NOT NULL	PK of the modified entity
OldValue	NVARCHAR(MAX)	Serialized pre-action state
NewValue	NVARCHAR(MAX)	Serialized post-action state

Column	Type / Constraints	Description
PerformedByUserId	INT, FK → Users.Id	Actor's user ID
Timestamp	DATETIME2 (UTC), NOT NULL	Precise UTC event timestamp

Chapter 9: Security Architecture & API Contracts

9.1 JWT Authentication Flow

HIMS uses stateless JWT-based authentication aligned with OWASP Top 10 guidelines, with specific mitigations for Broken Access Control (A01), Cryptographic Failures (A02), and Injection (A03).

21. User submits credentials to POST /auth/login; BCrypt constant-time comparison prevents timing attacks.
22. On success, JWT is constructed: claims sub (UserID), email, role; signed with HMAC-SHA256 (256-bit symmetric key).
23. Token returned with 2-hour expiry; client's auth.interceptor.ts injects it as Bearer header on every request.
24. ASP.NET Core Authentication Middleware validates signature, issuer, audience, and expiry per request.
25. Authorization Middleware enforces [Authorize(Roles="...")] attributes; returns HTTP 403 on privilege mismatch.
26. Angular role.guard.ts (CanActivate) verifies session token and role before any component initializes.

9.2 RBAC Permission Matrix

Endpoint / Feature	Customer	Agent	Officer	Admin
POST /auth/register	✓ Self	✗	✗	✗
POST /auth/addstaff	✗	✗	✗	✓
GET /quote/calculate	✓ Anon	✓	✓	✓
GET /policy/dashboard	✓ Own	✗	✗	✓
GET /staff/pending-policies	✗	✓ Assigned	✗	✓
PATCH /staff/verify-policy	✗	✓ Assigned	✗	✗
GET /staff/pending-claims	✗	✗	✓ Assigned	✓
PATCH /staff/decide-claim	✗	✗	✓ Assigned	✗
GET /staff/audit-history	✗	✓ Own	✓ Own	✓ All
POST /policy/create-template	✗	✗	✗	✓

9.3 Data Protection Standards

- All API communication enforced over HTTPS/TLS; plain HTTP disallowed in production.
- BCrypt work factor 12 provides resistance to GPU-accelerated brute-force attacks.
- JWT stateless design enables horizontal scaling without session synchronization.
- CORS policy restricts API access to explicitly whitelisted Angular origins.
- PII fields (FullName, Email) never written to application logs.

9.4 API Response Code Standards

HTTP Status	Trigger Condition	Example Scenario
200 OK	Successful read or non-creating mutation	GET /policy/dashboard returns policy list
201 Created	Successful entity creation	POST /quote/request creates PremiumQuote
400 Bad Request	Business rule violation	Claim exceeds CoverageAmount; pending documents on approval
401 Unauthorized	Expired or missing JWT token	Token past 2-hour expiry window
403 Forbidden	Valid token; insufficient role	Customer accessing /staff/pending-policies
500 Internal Server Error	Unhandled exception; intercepted by ExceptionMiddleware	Database connection failure

9.5 Core API Endpoint Catalogue

Method	Endpoint	Role	Description
POST	/api/auth/register	Public	Register new customer account
POST	/api/auth/login	Public	Authenticate and receive JWT
POST	/api/auth/addstaff	Admin	Create Agent or Officer account
GET	/api/quote/calculate	Public	Calculate premium (anonymous)
POST	/api/quote/request	Customer	Submit and persist quote application
GET	/api/policy/plans	Public	List available plan templates
GET	/api/policy/dashboard	Customer	Customer policy dashboard
POST	/api/policy/create	Admin	Create new plan template
PATCH	/api/policy/assign-agent/{id}	Admin	Assign agent to application
GET	/api/staff/pending-policies	Agent	Agent's verification inbox
PATCH	/api/staff/verify-policy/{id}	Agent	Approve or reject policy application
GET	/api/staff/pending-claims	Officer	Officer's claim settlement queue
PATCH	/api/staff/decide-claim/{id}	Officer	Approve or reject claim
POST	/api/claims/submit	Customer	Submit new medical claim
GET	/api/claims/my-claims	Customer	View own claim history
POST	/api/payment/process	Customer	Process payment; activate policy
GET	/api/staff/commissions	Agent	View commission earnings
GET	/api/staff/audit-history	Staff/Admin	Role-scoped audit log

Chapter 10: Frontend Architecture – Angular Design

10.1 Module & Component Structure

The Angular frontend uses a feature-module architecture with lazy loading for all role-specific sections, minimizing initial bundle size. The Core Module loads eagerly and provides application-wide services and guards.

Module	Lazy?	Key Components	Route
Core Module	No	Navbar, Sidebar, AuthGuard, RoleGuard, JWTInterceptor	/
Auth Module	Yes	LoginComponent, RegisterComponent	/auth
Customer Module	Yes	DashboardComponent, QuoteWizard, DocumentUpload, ClaimSubmit	/customer
Agent Module	Yes	PendingQueueComponent, DocumentReviewComponent, CommissionsComponent	/agent
Officer Module	Yes	ClaimQueueComponent, ClaimDetailComponent	/officer
Admin Module	Yes	StaffManagementComponent, PlanConfigComponent, AuditDashboard	/admin

10.2 Angular Service Registry

Service	Primary Responsibilities
AuthService	Manages JWT lifecycle in localStorage; provides currentUser() signal for role-based UI rendering.
CustomerService	Drives multi-step Quote Wizard; manages DocumentVault multipart/form-data uploads.
PolicyService	Fetches dashboard data, plan templates; handles policy assignment operations.
AgentService	Fetches verification queue; uses RxJS tap() to auto-refresh list on agent decisions.
OfficerService	Fetches claim queue; processes adjudication decisions; refreshes claim list on success.
PaymentService	Luhn-validated payment form; calls payment endpoint; handles invoice PDF download.

10.3 Quote Wizard – 6-Step Flow

27. Plan Selection: Customer selects Silver / Gold / Platinum plan tile.
28. Age Input: Validated Age ≥ 18; preview premium range displayed.
29. Quote Calculation: CustomerService calls GET /api/quote/calculate; premium/coverage shown with animation.
30. Account Confirmation: Anonymous users prompted to register; authenticated users proceed.
31. Document Upload: Multi-file uploader with type/size validation and progress indicators.
32. Submission: POST /api/quote/request; success state shows QuoteReference with live status tracker.

10.4 UI Design Principles & Validation Rules

UI Feature	Specification
Design Language	Glassmorphism — backdrop-filter blur, translucent panels, gradient backgrounds
Login Layout	Split-screen: 60% marketing content (hidden on mobile) / 40% auth portal
State Signals	Angular Signals (isLoading, isSuccess, isError) for instant UI feedback
Status Badges	Semantic color: Active/Approved = Green; Pending = Amber; Rejected = Red
Form Validation	Age > 18; payment card Luhn algorithm (client-side); file type/size enforcement
Responsive Design	Mobile-first; breakpoints at 768px (tablet) and 480px (phone)
Route Protection	CanActivate guard verifies JWT role before component initialization

Chapter 11: Audit Log Dictionary & Invoice Generation

11.1 Audit Log Compliance Standards

The PolicyActionLog is append-only by design — no entry is ever modified or deleted — providing a tamper-evident historical record for regulatory audit and forensic reconstruction. All timestamps are stored in UTC to eliminate timezone ambiguity.

11.2 Audit Action Type Dictionary

ActionType	Trigger	Fields Captured	Actor
PolicyVerification	Agent approves or rejects application	IsConvertedToPolicy: Old→New	Agent
ClaimDecision	Officer approves or rejects claim	Status: Old→New	Claims Officer
ClaimApprovalDeduction	Coverage deducted on approval	CoverageAmount: Before→After	System (Officer-triggered)
AgentAssignment	Admin assigns Agent to application	AgentId: null→UserId	Administrator
OfficerAssignment	Admin assigns Officer to claim queue	ClaimsOfficerId: null→UserId	Administrator
DocumentReview	Staff sets document status	Status: Pending→Verified/Rejected	Agent / Officer
PolicyActivation	Payment confirmed; policy goes live	IsPaid: false→true; Status→Active	System (Customer-triggered)
StaffOnboarded	Admin creates staff account	New UserId record	Administrator
PlanTemplateCreated	Admin creates plan	IsPlanTemplate: false→true	Administrator
CommissionRecorded	Commission auto-calculated	EarnedAmount: 0→computed value	System

11.3 Automated Invoice Generation (InvoiceService.cs)

The InvoiceService uses QuestPDF to generate branded PDF invoices, invoked exclusively by PaymentController after a confirmed payment. Generation and vault storage occur in a single synchronous operation within the payment transaction.

Section	Content	Data Source
Document Header	Company name, TransactionReference, PaymentDate	PaymentController parameters
Billed To	Customer full name and email	User.FullName, User.Email
Policy Details	SelectedPlanName, TierName, CoverageAmount	PremiumQuote entity
Transaction Table	Description, Period (1 Month), Rate, Total	PremiumQuote.CalculatedMonthlyPremium

Section	Content	Data Source
Footer	Page number, compliance disclaimer, issue date	System-generated

11.4 Invoice Storage Protocol

The generated PDF byte array is stored as a new DocumentVault entry: DocumentType = 'Invoice', Status = 'Verified' (no agent review required), RelatedEntityType = 'Policy'. This ensures every activated policy has an immediately accessible, immutable payment receipt available for customer download from their dashboard.

Chapter 12: Testing, QA & Non-Functional Requirements

12.1 Testing Strategy

Test Type	Framework	Scope	Owner
Unit Tests	XUnit + Moq	PremiumService calculation engine; boundary ages (18, 65, 99)	Dev Team
Integration Tests	XUnit + TestServer	Payment → Invoice → Vault pipeline; rollback on failure	Dev Team
API Contract Tests	Postman / REST Client	All 18 endpoints; happy path + error path validation	QA Team
Frontend Tests	Manual + Browser DevTools	Loading states, form validation, role guard redirects	QA Team
Security Tests	OWASP ZAP / Manual	JWT bypass, RBAC escalation, SQL injection attempts	Security Team

12.2 Premium Engine Test Cases

Test ID	Input	Expected Output	Category
TC-01	Age=18, Silver, Base=$500	Premium=$550, Coverage=$66,000	Boundary – Min Age
TC-02	Age=99, Silver, Base=$500	Premium=$950, Coverage=$114,000	Boundary – Max Age
TC-03	Age=30, Gold, Base=$500	Premium=$780, Coverage=$112,320	Normal – Gold Tier
TC-04	Age=45, Platinum, Base=$500	Premium=$1,050, Coverage=$189,000	Normal – Platinum Tier
TC-05	Age=17, Silver, Base=$500	Validation error: Age < 18	Boundary – Below Min
TC-06	Age=30, Invalid plan	HTTP 400: Plan not found	Negative – Invalid Plan

12.3 Performance Requirements

NFR ID	Requirement	Target
NFR-PERF-01	Premium calculation API response time	< 200ms at 99th percentile (100 concurrent users)
NFR-PERF-02	Policy dashboard load time	< 1 second for up to 50 active policies
NFR-PERF-03	Invoice PDF generation time	< 3 seconds per document
NFR-PERF-04	Concurrent user support	100 simultaneous authenticated users

NFR ID	Requirement	Target
NFR-PERF-05	Audit log query (last 10,000 entries)	< 500ms via SQL Server indexed query

12.4 Reliability & Availability Requirements

NFR ID	Requirement	Target
NFR-REL-01	System availability (excluding planned maintenance)	99.5% uptime per calendar month
NFR-REL-02	Data loss on unexpected shutdown	Zero loss for committed transactions
NFR-REL-03	Claim deduction atomicity	100% ACID; zero partial deductions permissible
NFR-REL-04	JWT validation failure handling	Graceful HTTP 401; no system crash or data exposure

12.5 Usability & Accessibility

- Supports Evergreen browsers: Chrome 120+, Edge 120+, Firefox 121+, Safari 17+.
- Fully responsive: mobile breakpoints at 768px (tablet) and 480px (phone).
- Inline form validation feedback within 100ms of field blur events.
- Status badges maintain WCAG 2.1 AA contrast ratio (minimum 4.5:1).

Chapter 13: Deployment, Scalability & Future Roadmap

13.1 Deployment Architecture

Component	Hosting Option	Minimum Spec
Backend API (.NET 8)	IIS on Windows Server 2022 or Kestrel	4 vCPU, 8 GB RAM, 100 GB SSD
Database (SQL Server)	SQL Server 2019+	8 vCPU, 16 GB RAM, 500 GB SSD (RAID)
Frontend (Angular)	IIS Static Hosting or CDN	Standard CDN; 1 GB storage
SSL/TLS	Reverse proxy (Nginx / IIS ARR)	Wildcard SSL; minimum TLS 1.2

13.2 Environmental Configuration

Critical per-environment configuration managed via appsettings.Production.json and Azure Key Vault:

- ConnectionStrings:DefaultConnection — encrypted SQL Server credentials.
- JwtSettings:SecretKey — 256-bit symmetric key; unique per environment; never in source control.
- JwtSettings:Issuer / Audience — must match deployed domain URLs.
- CORS:AllowedOrigins — exact Angular frontend URL(s) for the target environment.

13.3 Scalability Design

HIMS is designed for horizontal scalability. JWT stateless authentication means multiple API instances can run behind a load balancer without sticky sessions. EF Core No-Tracking queries on dashboard reads support read-replica offloading. The Angular build produces static artifacts distributable via CDN for global edge delivery.

13.4 Phase 2 & 3 Extensibility Roadmap

Phase	Feature	Technology	Business Driver
2	Azure Blob Storage for DocumentVault	Azure Blob Storage SDK	SQL storage relief; CDN document delivery
2	Automated Email Notifications	SendGrid API	Policy activation, claim approval/rejection alerts
2	AI OCR Document Scanning	Azure Form Recognizer	Reduce manual agent review time by ~70%
2	Real-Time Push Notifications	ASP.NET Core SignalR	WebSocket-based status updates without polling
2	Live Payment Gateway	Stripe / Razorpay SDK	Real card processing (currently simulated)
3	Risk Scoring Engine	ML.NET / Azure ML	Dynamic premium based on lifestyle & health data
3	Fraud Detection Model	Anomaly Detection API	Auto-flag suspicious claim patterns
3	Mobile Application	React Native / Flutter	Native iOS & Android customer portals

Phase	Feature	Technology	Business Driver
3	Multi-Currency Support	Currency Exchange API	International market expansion

13.5 API Versioning & Migration Strategy

URL-based versioning (/api/v1/, /api/v2/) will be implemented via ASP.NET Core API Versioning package as the system evolves. Version 1.0 endpoints remain frozen once v2.0 is released, with a minimum 6-month deprecation window. All new monetary schema columns must use DECIMAL(18,2); all new FK relationships must use DeleteBehavior.Restrict; all new audit-relevant entities must declare a corresponding ActionType in the log dictionary.

Chapter 14: Glossary, Acronyms & Revision History

14.1 Glossary of Terms

Term	Definition
Clean Architecture	Layered pattern enforcing unidirectional dependency flow: Domain ← Application ← Infrastructure ← API.
Coverage Amount	Remaining financial balance on a policy, decremented on each approved claim.
Deduction Gate	Business rule in claim adjudication blocking approvals that exceed the current CoverageAmount.
Document Gate	Business rule in policy verification blocking approvals unless all linked documents are Verified.
DocumentVault	Centralized binary document storage for both customer uploads and system-generated invoices.
IsConvertedToPolicy	Tri-state integer on PremiumQuote: 0 = Pending, 1 = Approved, 2 = Rejected.
MRR	Monthly Recurring Revenue — sum of all active policy MonthlyPremium values.
No-Tracking Query	EF Core .AsNoTracking() modifier returning entities without change-tracker attachment; reduces memory overhead.
Plan Template	Admin-configured insurance product (Silver/Gold/Platinum) with base premium and coverage parameters.
Transient Service	.NET DI lifetime creating a new instance per request, preventing shared state between callers.

14.2 Acronyms

Acronym	Full Form
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
DTO	Data Transfer Object
EF Core	Entity Framework Core
HIMS	Health Insurance Management System
JWT	JSON Web Token (RFC 7519)
MRR	Monthly Recurring Revenue
OWASP	Open Web Application Security Project
PII	Personally Identifiable Information
RBAC	Role-Based Access Control
SRS	Software Requirements Specification

Acronym	Full Form
TLS	Transport Layer Security

14.3 Revision History

Version	Date	Author	Summary
0.1 Draft	2025-01-15	Technical Lead	Initial scope, stakeholders, functional requirements Ch. 1–4
0.5 Beta	2025-02-10	System Architect	Added API contracts, database schema, state machines
0.8 RC	2025-03-01	QA Lead + Architect	Security review; testing and NFR chapters added
1.0 Final	03/12/2026	Technical Team	Full review; glossary, roadmap finalized; approved for release

— End of Document —

HIMS SRS v1.0 | ISO/IEC/IEEE 29148:2018 | © 2025 Health Insurance Inc.